



Benchmarking: Algoritmos de Ordenação

PPGCC02 – Estruturas de Dados

Docente: Prof. Dr. André Kawamoto

João Vitor Souza Rocha

Kaiky de Souza Lima

Campo Mourão, 2026

Descrição do Cenário

A implementação dos Algoritmos de Ordenação está disponível no repositório: <https://github.com/KaikyLima/Benchmarking-Algoritimos.git>. Os testes de Benchmark a seguir foram realizados em dois ambientes distintos. O Problema 1 foi realizado no Ambiente 1, enquanto os Problemas 2 e 3 foram realizados no Ambiente 2:

Ambiente 1:

- Sistema Operacional: Windows 11
- Processador: Intel(R) Xeon(R) w3-2423 2,11 GHz, 1 Socket, 6 Núcleos, 12 Processadores lógicos, Virtualização Habilitada
- Memória Cache: L1: 480 KB | L2: 12,0 MB | L3: 15,0 MB
- Memória RAM: 32,0 GB DDR5, 4400 MT/s, Fator de forma: DIMM
- Armazenamento: NVMe 3500 Micron 1024 954 GB
- GPU: NVIDIA T1000 8GB

Ambiente 2:

- Sistema Operacional: CachyOS Baseado em Arch Linux, Kernel 6.18 LTS
- Processador: 13th Gen Intel Core i5-13420H, Arquitetura Raptor Lake, 8 Núcleos / 12 Threads, Clock Min/Max: 400/4600 MHz
- Memória Cache: L1: 704 KiB | L2: 7 MiB | L3: 12 MiB
- Memória RAM: 16,0 GB DDR5 (2x 8 GB, Velocidade configurada: 5200 MT/s, Fator de forma: SODIMM)
- Armazenamento: SanDisk SSD Plus 1TB
- GPU: NVIDIA GeForce RTX 3050 Laptop GPU 6GB (Driver proprietário v610.43)

Problema 1 – Custo de Operações Teóricas vs. Tempo Real

O InsertionSort foi o mais rápido no vetor aleatório, com tempo de 0,535 segundos, contra 0,647 segundos do SelectionSort. A diferença está na lógica interna de cada algoritmo. O InsertionSort interrompe o laço interno assim que encontra um elemento já na posição correta. Em um vetor aleatório, isso significa que ele percorre, em média, apenas metade do trecho já ordenado por elemento inserido. O que explica as 225.697.282 comparações registradas, aproximadamente metade das realizadas pelo SelectionSort.

O SelectionSort, por sua vez, sempre percorre o trecho inteiro para encontrar o elemento mínimo, sem nenhum atalho. Por isso, suas comparações foram sempre exatamente 449.985.000, nos três cenários (aleatório, ordenado e inverso), confirmando que ele é completamente insensível ao estado dos dados.

Problema 2 – Desafio do Quicksort Clássico

Conforme a Figura 1, o algoritmo QuickSort obteve um tempo de execução consideravelmente maior do que o HeapSort. Isso se dá porque este algoritmo, que utiliza a técnica de Divisão e Conquista e, conforme a abordagem clássica, cujo pivô é sempre o último elemento do subvetor corrente, lida com seu pior caso ao encontrar um vetor totalmente ordenado. Neste cenário o pivô será sempre o maior elemento, o que causa desbalanceamento na partição (uma partição fica com $N-1$ elementos, enquanto a outra fica com 0). A recursão, que deveria dividir o problema pela metade, acaba apenas reduzindo o problema em uma unidade por vez. A complexidade degenera de sua média ideal para o pior caso: $O(N^2)$.

Já o HeapSort baseia-se na formação de uma árvore binária mapeada num vetor (*Heap*). Independente da ordenação dos dados de entrada e da subsequente remoção dos elementos, mantém-se uma estrutura de árvore balanceada com altura limitada a $\log N$, garantindo complexidade de $O(N \log N)$ para o melhor, pior e caso médio.

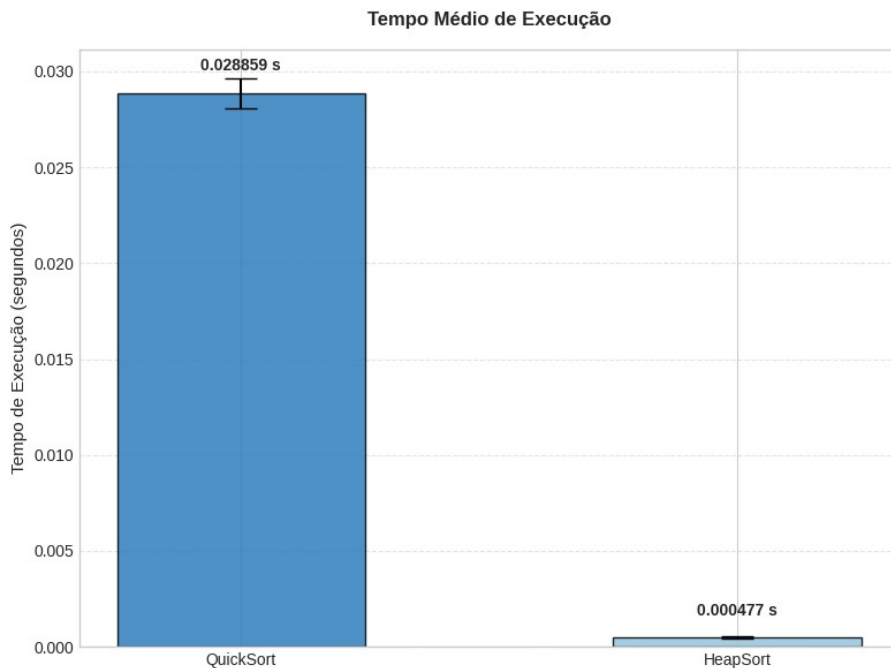


Figura 1: Tempo de Execução dos Algoritmos QuickSort e HeapSort

Problema 3 – Dados Quase-Ordenados e Shellsort

Ao gerar um vetor onde as trocas foram feitas apenas com vizinhos imediatos (apenas 0,5% de desordem), os elementos estão a no máximo uma ou duas posições de seu lugar correto. O algoritmo Insertion Sort, neste cenário, desempenha de forma satisfatória por ser capaz de interromper o laço de repetição caso detecte um elemento anterior menor. Dessa forma, não se faz necessário varrer o vetor pra trás, aproximando sua complexidade do melhor caso linear, ou seja, $O(N)$.

Já o Shellsort é caracterizado por realizar saltos longos entre os elementos do vetor, no entanto, como o vetor gerado não tem nenhum elemento muito longe do lugar correto, os primeiros passos do Shellsort (comparando o índice 0 com o 25.000, por exemplo) realizam milhares de comparações redundantes e que não realizam nenhuma troca. Isso é observado na Figura 2, que apresenta a comparação entre os tempos de execução para cada algoritmo, mas também na quantidade de trocas realizadas por ambos os algoritmos, que se manteve igual a 246 para ambos, em relação à quantidade de comparações:

- 50245 comparações para o Insertion Sort
- 700252 comparações para o Shell Sort

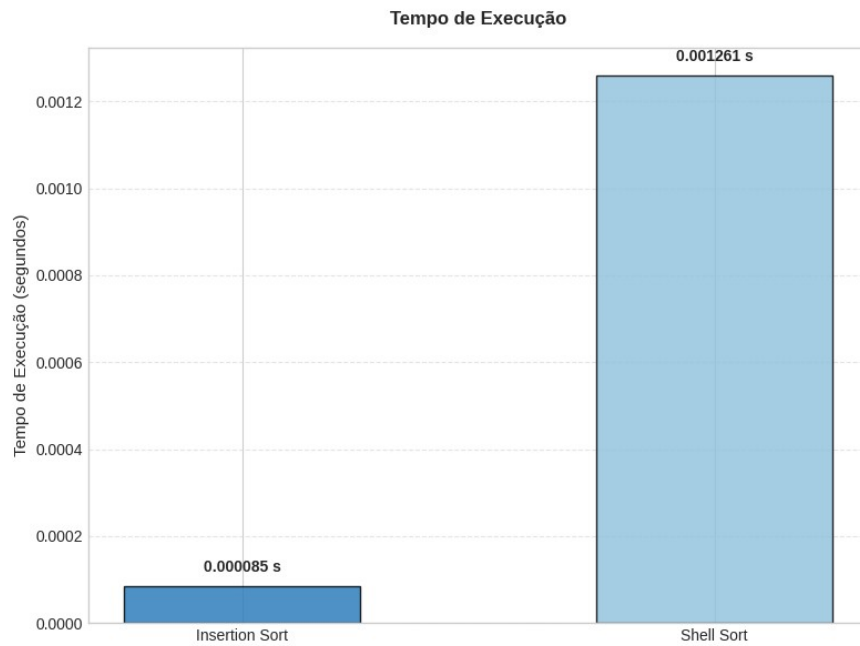


Figura 2: Tempo de Execução dos Algoritmos ShellSort e InsertionSort